Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier, Julie Bonnaire, and Marie-France Fourcade

Cognitive Neuroimaging Unit

CNRS INSERM CEA

Author Note

Christophe Pallier https://orcid.org/0000-0002-3324-666X Correspondence should be addressed to C. Pallier, Cognitive Neuroimaging Unit, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org. I would like to thank Luca Bonatti and Raphael Bordas for alpha testing this software. The authors have no conflicts of interest to disclose.

Abstract

We introduce `goxpyriment`, a new open-source software framework for programming behavioral and cognitive experiments using the Go programming language. The library is designed to address some limitations of existing Python-based experiment tools, particularly the runtime environment complexity that frequently complicates deployment across laboratories. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), `goxpyriment` compiles entire experiments into single, self-contained executable binaries with zero runtime dependencies. This drastically simplifies distribution to collaborators and testing computers. The programming interface, inspired by Expyriment (**krause2014**), was designed to be human friendly. The library includes an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, ...) and audio capabilities (WAV playback and tone generation). While developping `goxpyriment`, we focused on timing reliability. Input events are timestamped by the operating system at hardware-interrupt time, so reaction times are computed by subtracting two OS-level timestamps rather than relying on continuous polling. Go's garbage collector can be disabled, greatly reducing the probability of unpredictable pauses that could corrupt stimulus timing. Finally, a set of over forty psychology experiments implemented in `goxpyriment` are provided that promote not only learning by humans but also improve the ability of modern AI-assisted coding tools to help program experiments. The framework is released under the GNU General Public License v3 and is freely available at https://github.com/chrplr/goxpyriment.

Keywords: framework, library, stimulus presentation, timing, behavioral experiments, psychophysics, psychology

Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Introduction

Experiment-control software occupies a central position in behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording, the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through two libraries: Expyriment (**krause2014**) and PsychoPy (**peirce2019**). Before Python, MATLAB with the Psychophysics Toolbox (**brainard1997**; **kleiner2007**) was the de facto standard in many laboratories. More recently, web-based frameworks such as jsPsych (**deleeuw2015**; **deleeuw2023**) and lab.js (**henninger2022**) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB, extended with specialized C/C++ libraries (e.g. the Psychtoolbox), can offer good timing on well-configured hardware. Yet Psychtoolbox scripts written on Windows often do not work out of the box on Linux (and vice-versa). MATLAB can also be quite expensive when one cannot purchase academic licenses (as is the case for non diploma-delivering research institutions). Python tools provide a more open, flexible programming environment with a large scientific ecosystem. Python-based experiments can achieve high-precision timing, for example by relying on the psychtoolbox Python module which provides pieces of the Psychtoolbox-3 (see https://pypi.org/project/psychtoolbox/). However, the use of Python introduces practical difficulties. First, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (conda, pip, venv) and can be a source of breakage across operating-system versions. For example, installing the `psychopy` module under Linux can crashes while attempting to compile the wxPython package when there is no precompiled wheel. Second, Python's garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording

during the critical trial loop. Web-based tools trade timing precision for accessibility: browser-mediated stimulus presentation is subject to event-loop latency that makes sub-frame timing difficult or impossible (**bridges2020**; **anwylirvine2021**).

The Go programming language (**donovan2016**) offers a different perspective. Designed from the ground up for simplicity, Go has fewer moving parts than Python; as a consequence, the code is often described as "boring" but highly predictable, as the language structure typically enforces only one way to solve a given problem. In addition, Go is strongly typed and encourages error checking, which translates into less run-time debugging. Another important feature of Go is that it (cross-)compiles to native machine code for each target platform and can produce self-contained binaries that embed the stimuli, experiment lists, and thus have no runtime dependencies. Its garbage collector is concurrent and can be suspended for the duration of a timing-critical section. These properties make it an attractive basis for a behavioral experiment library that prioritizes timing reliability and deployment simplicity.

The present article describes the design and capabilities of `goxpyriment`, discusses its timing architecture, and situates it relative to existing tools. Specifically, we highlight three main contributions of this framework: (1) zero-dependency deployment via single compiled binaries that simplify distribution across laboratory hardware; (2) native high-precision timing achieved directly through the Go and SDL3 APIs without requiring external C/C++ helper engines; and (3) a linear, straightforward API that is well suited for modern AI-assisted coding workflows.

Framework Design

Programming Model

Every experiment centers on an `Experiment` object that manages the window, renderer, and audio subsystems. Execution is wrapped in a `Run` callback that provides a safety layer: if the participant presses the Escape key or closes the window, the library catches the resulting internal signal, safely flushes the data file to disk, and shuts down the SDL subsystems. This ensures that data is preserved even if a session is aborted prematurely.

It is possible, but not necessary, to build the experiment around a hierarchical structure of blocks and trials, over which the program can iterate, following a patten inspired by Expyriment (**krause2014**). Each trial is a collection of factors that can be randomized and logged. Data collection is handled by an `exp.Data` object, which writes to a CSV file withal metadata headers providing extensive information about the computer OS and hardware, timestamps, etc.

`goxpyriment` provides an optional graphical `GetParticipantInfo` dialog that opens before the main experiment window (see Fig. **??**). This allows the experimenter to input subject IDs, select experimental conditions from discrete menus, and toggle settings like fullscreen mode. These values are automatically recorded in the output data file header.

Single-Binary Deployment (Zero Dependencies)

Go compiles programs into a single statically linked executable with no external runtime dependencies. Crucially, the `go-sdl3` package embeds the pre-built SDL3 runtime libraries (**sdl3**) directly into the executable so that the user do not need to install any library by herself. In addition, the stimuli, experimental lists, fonts, and other assets can be embedded in the binary too. As a result, distributing an experiment requires copying just one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of installing experiments on multiple machines or sharing materials with collaborators.

Explicit Garbage Collection Control

While the garbage collectors in languages like Python or JavaScript can introduce unpredictable pause times that corrupt timing, Go exposes a runtime API for suspending the garbage collector. `goxpyriment` uses this mechanism in its timing-critical loops, ensuring that stimulus onsets and offsets are not delayed by memory management tasks.

Hardware-Synchronized Presentation

Through go-sdl3 (**gosdl3**), `goxpyriment` relies on the Simple DirectMedia Layer (SDL, version 3), a cross-platform development library designed to provide low-level access to audio, keyboard, mouse, joystick, and graphics hardware (**sdl3**). For example, `screen.Update()` calls SDL's `SDL_RenderPresent`. When VSYNC is enabled, this function blocks until the display's vertical retrace. When VSYNC is disabled, `SDL_RenderPresent` returns immediately, supporting variable refresh rate (VRR) monitors.

High-Precision Streams

For paradigms with dense, high-speed stimulus sequences (e.g., retinotopic stimulation), `goxpyriment` provides specialized stream functions such as `PresentStreamOfImages`. These functions disable garbage collection, drain the event queue, and busy-wait at the sub-frame level to ensure that each stimulus onset is scheduled accurately. A `TimingLog` records the target and actual onsets for post-hoc verification, as well as the timing of keys pressed during the presentation of the stream.

OS-Level Response Timestamps

Response timing in `goxpyriment` leverages SDL3 OS-level event timestamps. The `GetKeyEventTS` method returns the `KeyboardEvent.Timestamp` field, which is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). Because SDL events carry their timestamp in the event structure itself, the event remains in the queue with its original timestamp regardless of how much computation occurs between stimulus onset and response collection. By comparing the response timestamp with the `Screen.FlipTS()` timestamp—which uses the same clock—reaction times can be measured with minimal jitter and no interpreter overhead (of course, potential hardware level lags at the level of the keyboard or mouse can still be present). The big advantage of this approach is there there is no need to continusously pool the HID device, e.g. keyboard or mouse, to detect events. Thus, a complex sequence of stimuli can be programmed and all events will be available to process later, when needed.

Triggers

Generic functions for reading from and writing to parallel and serial ports are provided. In addition, the current version implements TTL input/output interfaces for two specific USB-serial devices: the DLP-IO-8 (see https://github.com/chrplr/dlp-io8-g) and an Arduino Mega 2560 utilizing a custom protocol (see https://github.com/chrplr/neurospin-meg-ttl-box).

While USB latency can be a concern, it may be mitigated under Linux by setting the value of `/sys/bus/usb-serial/devices/ttyUSB0/latency_timer` to 1. To avoid USB entirely, one can use an Ethernet-based interface such as the LabJack T4 (https://labjack.com/products/labjack-t4), which can be controlled via Go's `modbus` library (https://github.com/goburrow/modbus)."

The Timing Verification Suite

The repository includes a dedicated timing verification suite (`tests/Timing-Tests/`) that allows researchers to characterize their hardware's performance with external devices. The suite includes tests for measuring frame-interval jitter, verifying single-frame flash reliability via photodiode, and calculating audio-visual SOA latencies. Other tests measure hardware-buffer pipeline latency for audio and characterize the USB-serial jitter of trigger devices.

Table **??** presents results of some of these tests running on a modest Raspberry PI 4 with 2Gb of RAM. The Raspberry displayed through HDMI on a Dell Monitor 1905FP, set at a resolution of 1280×1024 and a refresh rate of 60Hz. The only performance tweak applied was to set the timer pooling for usb_serial to the minimum possible value. A Black Box Toolkit version 3 (**plant_2004**), controlled by our software (**Pallier_bbtkv3**), captured input events on the 'Opto1', 'Mic1' and 'TTLin1' lines. The first lines show the duration and inter-onset timings. The numbers are quite stable for the 'Opto' and 'TTLin' sensors (SD<0.5ms), and more variable for the 'Mic1' (sound detector). We believe that the variability is more likely to come from the loudspeaker than the actual playout by the computer. The last two lines of Table **??** reports the delays (lags) betwen the TTL signal and the video signal, and

between the TTL and audio stimulus. These measures reveal a lag of 38ms, corresponding to a bit more than 2 frames at 60Hz, for the visual stimulus, and a lag of 70ms for the audio audio stimulus. As these lags are relatively stable (with standard-deviations of 0.17ms and 2.77ms respectively), a user should compensate for them in an actual experiment. These lags will vary with the video and audio hardware and software (drivers & servers) and must be assessed for each configuration. A special folder in the goxpyriment github repository (`performance-reports`) is dedicated for users to share the results of timing tests.

AI-Assisted Coding

An AI-Friendly framework

Go's design emphasizes simplicity and predictability. Because the language has few synonymous constructs, it is particularly well-suited for use with AI-powered coding assistants, for example Claude Code or Gemini cli. The framework's AI-friendliness also comes from other primary technical merits:

Type Safety. Unlike dynamic languages where errors like misspelled method names or invalid argument types (e.g., supplying a string "red" where a RGB color value is expected) only surface at runtime, Go's compiler catches these during the first build attempt. The strict typing reduces erroneous API calls in AI-generated code.

Linear Control Flow. In some other frameworks, the programmer must manage an explicit event loop (e.g., `while True`), manually polling for events and checking for exit signals. AI Coders can miss these steps, leading to non-responsive windows or missed reation times. In `goxpyriment`, the `exp.Run` wrapper handles the event loop and automatic cleanup internally, simplfting the work of both the human and AI coder.

Asset Embedding: A frequent failure point for AI-generated experiments is incorrect file paths for stimuli. Using Go's `//go:embed` directive, an AI can generate code that bundles assets directly into the binary, eliminating path errors caused by mismatched directory structures.

It would be worth conducting a study examining the AI coders abilities to

program psychology experiments with different frameworks and programming langages, but to perform such a study scientifically goes largely beyond the scope of the present paper.

A Cautionary Note

As we just argued, `goxpyriment` is well-suited for AI-assisted coding: its API and the provided examples give strong guidance to AI coding agents prompted to program psychology experiments. Someday experiments from the examples gallery were actually completely entirely generated by Claude or Gemini (see the Author Contributions section). Yet, even if these agents are extremely apt and quick at coding, human supervision remains clearly necessary, as illustrated in Fig. **??**.

Comparison with Existing Tools

Table **??** summarizes key properties of `goxpyriment` alongside five widely used experiment frameworks.

`goxpyriment` resembles Expyriment in its API design: a central experiment object, a `present()`-based stimulus API, a structured trial-loop model, and output to `.csv`-format files with metadata. The code for a simple experiment (Parity Decision) is provided in Appendix (Listing 1). The primary technical differences are the use of Go rather than Python and the use of SDL3 rather than the older Pygame/SDL2 stack. From the user's point of view, `goxpyriment` adds *stream* objects which, as their name indicates, allow displaying sequences of stimuli with tightly controlled timing.

Example Experiments

The repository provides over forty demos and examples of classic psychology paradigms, including the Stroop task, Attentional Blink (RSVP), Masked Priming, Retinotopic mapping for fMRI, and adaptive threshold estimation using the QUEST algorithm. These examples serve as both functional templates and a gallery of the framework's capabilities.

Limitations and Future Directions

Video playback. The video playback functions currently do not expose frame-accurate timestamps or hardware trigger outputs. Adding precise onset timestamps and trigger support during video playback is our highest priority for the next release, as these capabilities are required for EEG and fMRI paradigms that include video stimuli.

Eye-tracker support. There is none for the moment. This is a target for future releases.

Audio and Video recording. There is currently no support for recording audio responses and measuring naming times, nor videos. As SDL3 supports recording audio and vido, these features could be implemented in the future.

Online deployment. `goxpyriment` does not currently support web-based deployment. In theory, this should be possible as Go can compile to web assembly (wasm) and SDL3 supports EMSCRIPTEN (https://emscripten.org/). However, when we attempted it, we realized that the various tools involved are not yet mature enough.

Language barrier. Researchers who are unfamiliar with Go, that is, the vast majority, will need to invest a bit of time in getting acquainted with the language. To this end, many web resources exist, for example, https://go.dev/tour/ and https://gobyexample.com/. As Go is a descendant of C, the syntax will be familiar to people who know a language of the same family, e.g., Java or Javascript. Finally, the examples and the migration guide we provide at https://chrplr.github.io/goxpyriment, are also meant to ease the transition.

Name of the project. The name "goxpyriment" is quite hard to pronounce and memorize.

Listing 1: Example of a Parity Decision experiment

```
package main

import (
        "slices"
        "strconv"

        "github.com/chrplr/goxpyriment/control"
        "github.com/chrplr/goxpyriment/design"
        "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
        exp := control.NewExperimentFromFlags("Parity Decision",
                                control.Black, control.White, 32)
        defer exp.End()

        Targets := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
        NTrialsPerTarget := 2
        EvenResponse     := control.K_F
        OddResponse      := control.K_J
        Instructions := "When you'll see a number, your task to decide,
                as quickly as possible, whether it is even or odd.\n\n
                if it is even, press 'F'\n\nif it is odd, press 'J'"

        cue := stimuli.NewFixCross(25, 2, control.DefaultTextColor)
        // creates a map number -> image
        Image := make(map[int]*stimuli.TextLine)
        for _, num := range Targets {
                Image[num] = stimuli.NewTextLine(strconv.Itoa(num),
                                0, 0, control.DefaultTextColor)
        }

        trials := slices.Repeat(Targets, NTrialsPerTarget)
        design.ShuffleList(trials)

        exp.AddDataVariableNames([]string{"number", "key", "rt",
                                                "correct"})

        exp.Run(func() error {
                exp.ShowInstructions(Instructions)

                for _, t := range trials {
                        exp.Blank(1000)
                        exp.ShowTimed(cue, 500)
                        key, rt, _ := exp.ShowAndGetRT(Image[t],
                                []control.Keycode{EvenResponse, OddResponse}
                                -1)
                        correct := (t%2 == 0) == (key == EvenResponse)
                        exp.Data.Add(t, key, rt, correct)
                        if !correct {
                                exp.Audio.PlayBuzzer()
                        }
                        exp.Wait(500)
                }

                return control.EndLoop
        })

        exp.ShowEndMessage("Experiment complete. Thank you!\n\n
                        Press any key to exit.")
}
```

Declarations

Funding

No funding was received to assist with this project nor with the preparation of this manuscript.

Conflict of interest/Competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Availability of data and materials

Code availability

The source code of `goxpyriment` is available under the GNU General Public License v3 at https://github.com/chrplr/goxpyriment. All example experiments described in this article are included in the repository. No empirical data are reported in this article.

Authors Contributions

Christophe Pallier: Conceptualization, Software implementation, Documentation and Paper Writing, with assistance from **Claude Sonnet 4.6** and **Gemini 2.5**. Consistent with current editorial norms (**natureportfolio2023**; **springernature2023**), Claude and Gemini are listed under Author Contributions rather than as a named author, because authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content. **Julie Bonnaire** and **Marie-France Fourcade** set up the timing measurement system and performed tests.

Table 1

Results of a timing test performed on a Raspberry Pi 4 (Linux, arm64). Every half second, a 200ms tone was played, a 200ms white screen was displayed, and a TTL signal was sent. 1000 cycles. Measurements were performed with a Black Box Toolkit v3.

	Min	P5	P25	P50	P75	P95	Max	Range	P95-P05	Mean	SD
Durations[1]											
Opto1	220.0	220.2	220.2	220.5	220.5	220.8	220.8	0.8	0.5	220.4	0.17
Mic1	220.5	220.5	220.8	220.8	223.8	230.2	254.5	34.0	9.8	222.8	3.73
TTLin1	5.0	5.2	5.2	5.2	5.5	5.5	6.2	1.2	0.2	5.3	0.13
Inter-onsets intervals											
Opto1	499.5	499.5	499.8	499.8	500.0	500.2	500.2	0.8	0.8	499.9	0.19
Mic1	482.5	494.0	497.2	500.2	502.0	506.0	518.2	35.8	12.0	499.9	3.79
TTLin1	493.5	499.5	499.8	499.8	500.0	500.0	500.5	7.0	0.5	499.9	0.27
Paired-Event Onset Differences relative to TTLin1											
TTLin1→Opto1	37.8	38.2	38.2	38.5	38.5	38.8	39.0	1.2	0.5	38.5	0.17
TTLin1→Mic1	62.0	66.5	69.0	70.8	72.6	75.5	87.8	25.8	9.0	70.9	2.77

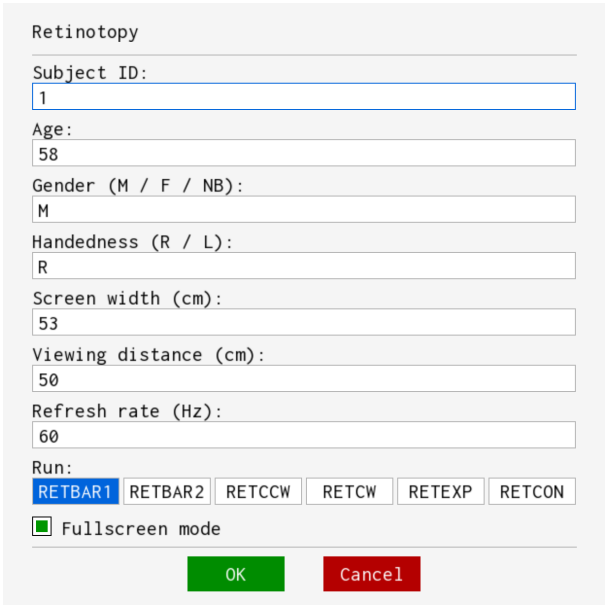
[1] Because smoothing was set on the 'Opto' and 'Mic' detectors, 20ms must be subtracted from these durations, as per the BBTK User Manual.

Table 2

Comparison of selected experiment frameworks.

Property	goxpyriment	Expyriment	PsychoPy	PTB (MATLAB)	jsPsych
Language	Go	Python	Python	MATLAB	JavaScript
Deployment	Single binary	pip/conda	Standalone or pip	MATLAB license	CDN / npm
GC control	Explicit disable	No	No	N/A	No
VSYNC sync	Yes (SDL3)	Yes (SDL2)	Yes (Pyglet/PTB)	Yes	No
VRR support	Yes	No	No	Partial	No
Hardware triggers	Yes	Yes	Yes	Yes	No
Response device API	Unified	Device-specific	Device-specific	Device-specific	N/A
Event timestamp	SDL3 hardware	pygame timer	iohub/PTB clock	`GetSecs` (μs)	Browser clock
Stimulus set	Moderate	Moderate	Extensive	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No	No
Online deployment	No	No	Partial (PsychoJS)	No	Yes
Reference	This paper	**krause2014**	**peirce2019**	**brainard1997**	**deleeuw2023**

A.



B.

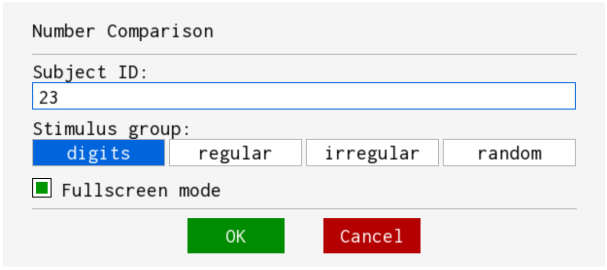


Figure 1

Examples of `GetParticipantInfo` *dialogs for* Retinotopy *(A.) and* Number-Comparison *(B.), two paradigms provided in the examples accompanying the library. These UIs makes it practical for the experimenter to choose among a small set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager. Note that which fields appear in the UI is configurable. It is also prossible to pass the information as arguments on the command-line and skip these UIs.*

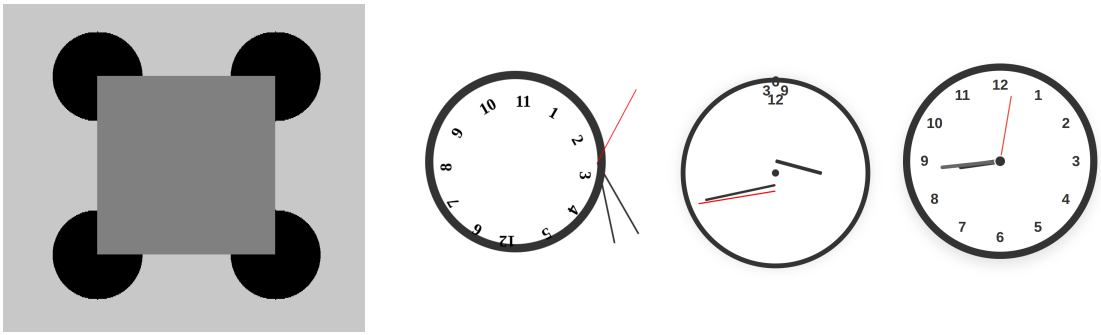


Figure 2

AI errors: *Left: asked to write a program displaying Kanizsa's illusory rectangle, an AI produced the result shown, explaining: "I have changed the Kanizsa example to make things robust and more visible [. . .] A darker gray rectangle is drawn at the center, clearly visible against the light-gray background." Right: output from the AI clock project where AIs are asked to write programs drawing a clock; note the correct response on the right (see https://clocks.brianmoore.com/ for more).*